LLM-Based Test Generation: Information Sources, Generation Strategies, and Quality Evidence

ANONYMOUS AUTHOR(S)

Large language models are increasingly used to generate test scenarios, executable test suites, assertions, interaction sequences, and fuzzing infrastructure. These artifacts serve different purposes and rely on different sources of information about correct behavior. A test that increases implementation coverage, a test that agrees with a reference program, and a test that detects a requirements violation therefore provide distinct kinds of evidence. We present a structured narrative survey that connects the generation process to the evidence used to justify test quality. Drawing on 95 curated source records through 29 September 2026, we organize the field along four dimensions: testing objectives and artifacts, information available during training and generation, generation and learning mechanisms, and evaluation evidence. We synthesize work on unit and requirements-based testing, test oracles, API and GUI testing, fuzzing, and learned test generators. The resulting framework explains how execution feedback can improve executability while also affecting the independence of an oracle, why specification provenance and training-time reference supervision matter for comparisons, and how downstream code-selection gains differ from test correctness. We use these distinctions to organize benchmarks and derive a research agenda for independent oracle assessment, budget-aware generation, repository-scale evaluation, and test maintenance.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Natural language generation.*

Additional Key Words and Phrases: large language models, test generation, test oracles, software testing, survey

ACM Reference Format:
Anonymous Author(s). 2027. LLM-Based Test Generation: Information Sources, Generation Strategies, and Quality Evidence. In *Proceedings of FSE 2027*. ACM, New York, NY, USA, 22 pages.

1 Introduction

Test generation requires a connection between what software should do and observations that can expose when it does something else. Constructing that connection involves several kinds of work: interpreting requirements, selecting inputs, arranging an execution environment, choosing observations, and expressing an expected result. Large language models (LLMs) can contribute to each of these activities because they operate across natural language, program code, and semi-structured technical documentation. Research has consequently expanded from generating unit-test code to producing acceptance scenarios, postconditions, API call sequences, and fuzzing drivers [75, 85, 91].

This breadth creates a comparison problem. Two approaches can both generate executable tests while having different access to the system under test (SUT), different sources of expected behavior, and different intended uses. A regression generator may inspect an implementation and retain tests that pass on it. A requirements-driven generator may need to produce an oracle before an implementation exists. An issue-reproduction agent may receive a report of an already observed failure. The same passing test carries a different meaning in each setting. Comparing their coverage or execution success without describing these conditions obscures what has actually been learned about test quality.

TestGen-LLM filters regression improvements through execution and coverage [3]; CoverUp uses coverage to guide generation [57]; PyTester trains description-conditioned generation using

FSE 2027, Shenzhen, China
2027.

reference execution [70]; UTRL couples code and test generators through adversarial learning [36]. These methods differ in when behavior becomes available and how it influences tests.

Existing surveys provide substantial foundations. Broad reviews map LLM applications across testing and repair [5, 75]; unit-testing surveys organize techniques, model adaptation, and quality assurance [11, 91]; requirements-based reviews cover transformations from natural-language and formal specifications [19, 85]. A recent systematic review directly addresses LLM-based test-case generation [71]. Oracle generation, hybrid techniques, and evaluation limitations already appear in these accounts. Our organizing question is how these aspects interact: *what information produces a test, and what evidence justifies using that test to judge software?*

We develop this question through a structured narrative synthesis of a curated collection of 95 source records. The collection includes seven surveys and roadmaps, together with primary methods, benchmarks, empirical studies, and selected neural predecessors. Its breadth supports a cross-setting analysis; its preliminary screening status is described in Section 3. We make three contributions. First, we define a framework that separates testing objectives, generated artifacts, information access, and quality evidence. Second, we use the framework to connect prompting, program analysis, feedback, agents, and model training across testing settings. Third, we organize evaluation designs according to the claims they can support and derive concrete research questions from the remaining mismatches.

The central argument is that test generation should be understood as both an artifact-construction problem and an evidence-construction problem. Better syntax and broader execution improve the first. Confidence in expected behavior requires the second. This distinction helps explain when tools from different research traditions can be combined and which claims require additional validation.

2 Foundations and Analytical Framework

2.1 What Is Generated?

An executable test combines a setup, an input or action sequence, and an oracle that judges the resulting behavior. Its setup can include fixtures, mocks, dependencies, credentials, database state, and cleanup. Its input may be a function argument, an HTTP request, a GUI action, or an entire generated program. Its oracle may check an exact output, an exception, a relation across executions, or a safety property. A test suite additionally determines which tests coexist, how state is isolated, and when execution ends.

Many generation methods produce an intermediate artifact. A natural-language scenario still requires operationalization. A grammar defines a family of admissible inputs but does not necessarily determine their expected outputs. A fuzzing driver connects generated data to a target API while relying on a separate failure detector. We therefore distinguish *test design*, *test input*, *test oracle*, *executable test*, and *test-generation infrastructure*. This distinction preserves the contributions of input-only and infrastructure methods without attributing complete test synthesis to them.

2.2 Testing Objectives Determine Oracle Roles

Functional conformance asks whether behavior satisfies an external requirement. Regression testing asks whether a change violates behavior that should be preserved. Bug reproduction asks whether a reported failure can be exercised and distinguished from its repair. Fuzzing often searches for crashes, sanitizer violations, or disagreement among implementations. Tests used in code generation may instead rank candidate solutions or provide debugging feedback. These objectives can share execution machinery while requiring different oracle evidence.

Consider a requirement that a function return a sorted permutation of its input without modifying the input. A test may assert the sorted result yet omit the mutation constraint. It can execute

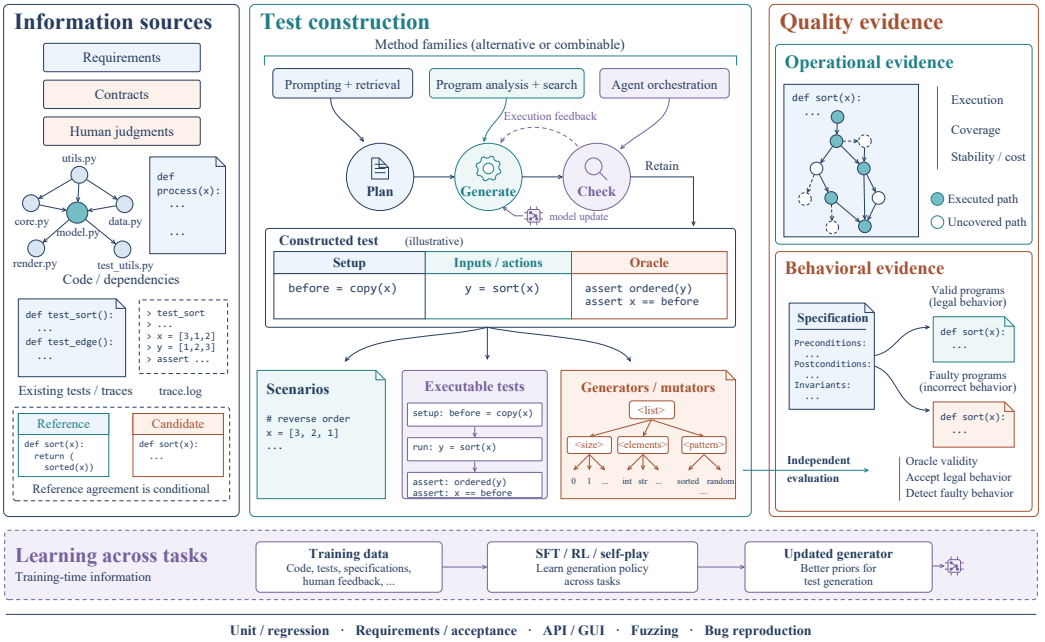


Fig. 1. A synthesis of LLM-based test generation. Information sources support planning, generation, and checking; alternative or combined mechanisms construct scenarios, executable tests, and generators. The sorting sketch illustrates setup, action, and a partial oracle. Dashed feedback revises the current artifact; matching violet model icons connect the cross-task learning output to generation. Operational and behavioral evidence address different quality properties. Independent evaluation supplies held-out evidence without feeding its results back into generation.

successfully and reach every line while leaving part of the requirement unchecked. A test copied from an implementation that sorts in place can even preserve the very violation that a conformance test should reveal. Conversely, a test that expects one particular ordering among equal-key objects may reject a legal implementation if stability was never required. This illustrative example motivates evaluating both detection of invalid behavior and acceptance of permitted behavior.

2.3 Information Access and Correctness Evidence

We distinguish three stages of information access: training, generation, and final evaluation. At each stage, relevant sources include requirements, focal code, repository context, existing tests, reference implementations, candidate implementations, execution traces, and human decisions. Access is a property of an experimental variant rather than an entire paper. For instance, inference can use a description and signature even when training rewards require a reference program, as in PyTester [70]. Describing that configuration simply as “code-free” would erase a material dependency.

An information source can play more than one role. Source code helps resolve imports and construct objects; the same code can also supply expected outputs. A static dependency graph can identify callable APIs without validating their semantics. Execution of a trusted reference provides a concrete output under the reference assumption. Execution of the candidate under test provides

an observation about that candidate. These roles should be recorded separately because context relevance and oracle independence answer different questions.

Specification provenance adds a fourth consideration alongside the three access stages. A docstring generated from buggy code remains implementation-derived, even if the next generator receives only that docstring [95]. Conversely, a requirements-oracle study gives the generator requirements and minimal method structure, while researchers construct those requirements from buggy/fixed differences [46]. Generation permissions and dataset-construction permissions are therefore separate fields. A PR's stated intent provides another behavioral source, interpreted alongside the change and before/after executions in Testora [58]. These sources can all be useful, but their independence from the behavior under examination differs.

Figure 1 connects the four dimensions of our analytical framework: objectives and artifacts, information access, generation mechanisms, and quality evidence. The dimensions are intentionally orthogonal. "Agent-based" describes how decisions are coordinated; "requirements-based" describes an information source; "unit testing" describes a testing level. Treating these labels as competing branches of one taxonomy would place overlapping studies in arbitrary categories. We instead use testing settings to organize the landscape and mechanisms to explain recurring design choices.

3 Review Scope and Synthesis Method

3.1 Research Questions

Six questions guide the synthesis and connect its sections:

RQ1: Objectives and artifacts. Which testing objectives, systems, and levels are addressed, and what artifacts are generated? Section 4 maps the settings and outputs.

RQ2: Information and oracles. What information is available during training and generation, and where does evidence of correct behavior originate? Sections 2 and 6 distinguish access stages and sources.

RQ3: Generation strategies. How are LLMs combined with retrieval, analysis, execution, search, and agents? Section 5 compares these mechanisms.

RQ4: Learning objectives. What do training data, rewards, and adversarial curricula optimize? Section 6 examines learned generators.

RQ5: Evaluation evidence. Which quality claims do benchmarks support, and under what conditions are results comparable? Section 7 analyzes protocols and measurements.

RQ6: Practical use and open problems. What is supported by development-workflow evidence, and which capabilities still need validation? Section 8 connects adoption, maintenance, and research directions.

3.2 Collection and Selection

This survey is a structured narrative synthesis of a preliminary literature collection assembled through 29 September 2026. Discovery used public web search, primary publication pages, arXiv records, author-hosted resources, and references identified through existing surveys. Query families combined large language models with unit-test generation, requirements-based testing, oracle generation, API and GUI testing, fuzzing, reinforcement learning, and evaluation benchmarks. Named-method searches expanded these families to identify related versions and recent developments. A supplemental pass added 35 records through topic-specific queries and targeted tracing of review bibliographies; its search notes and source-specific inspection records accompany the register. The source register contains 95 records, including seven surveys and roadmaps and 88 method, benchmark, empirical, and historical records; these counts describe the curated register rather than an exhaustive population of studies.

Table 1. Existing review families and the emphasis of this synthesis. These are differences in scope and organization, not claims that individual topics were previously absent.

Review family	Established coverage	Emphasis here
Broad testing surveys and roadmap [5, 75]	Testing tasks, LLM techniques, challenges, repair-related applications	Connect generated artifacts to their oracle evidence
Unit-testing surveys [11, 91]	Generation methods, model adaptation, quality assurance, lifecycle concerns	Compare code-conditioned and specification-conditioned settings
Requirements-based reviews [19, 85]	Requirements representations, transformations, industrial use	Separate scenario design, input construction, and executable oracles
LLM test-case SLR [71]	Techniques, data, metrics, preprocessing and postprocessing	Track information access across training, inference, and assessment

We include methods producing testing artifacts and studies evaluating their generation. Tests for debugging and code selection are included when they clarify test quality's downstream role. General code generation and testing LLMs themselves fall outside the central scope. Selected neural predecessors provide context [15, 73]. Public primary sources were preferred. Retrieval excluded MDPI sources and records with unclear provenance, limiting coverage.

Related preprint and publication versions were treated as a study family. A changed title alone was not counted as a new method. The bibliography retains the inspected primary source and uses verified publication metadata where available. Several records remain preprint or author-resource citations even when their accompanying register reports an accepted venue; final proceedings normalization is incomplete. Publication status and evidence strength remain separate: a peer-reviewed abstract can establish a method's broad objective, while its detailed information access may still require inspection of the full text.

3.3 Extraction and Interpretation

The register records artifacts, information sources, mechanisms, analytical use, and inspection depth. Each entry was checked against a primary page or document. Most checks concern metadata and abstracts; selected passages establish information access, reward supervision, and metric definitions. We do not pool effect sizes or rank methods. Independent dual screening, inter-rater validation, and experimental reproduction have not been performed.

We distinguish method descriptions, interpretations of measured properties, and evaluation recommendations. Reference checking describes a method; reference consistency interprets its evidence; testing alternative legal implementations extends the protocol. Recommendations are not reported findings.

AI assistance was used to expand search terms, organize retrieved literature, propose the cross-study categories, draft the synthesis, and design its conceptual figures. The accompanying register preserves source links and inspection depth. The present coding has not received independent human validation. This status is relevant because category assignments can affect the survey's conclusions even when individual bibliographic records are correct.

3.4 Relationship to Existing Reviews

The updated unit-test review by Chu et al. reports 178 publications eligible for full-text review through July 2026 in its September 2026 revision [11]. Its count and this survey's 95-record register

Testing setting	Scenarios / obligations	Inputs / actions	Tests / oracles	Generators / mutators
Unit / regression			TestPilot [P,F] CoverUp [P,F]	
Requirements	RAGTAG [P,R]		Test Flow [P]	
Oracle synthesis			nl2postcond [P] Nexus [P,F]	
API / GUI		DroidAgent [P,F]	SAINT [P,A]	
Fuzzing		TitanFuzz [P]		PromptFuzz [P,F] ELFuzz [P,S]
Code-task testing			PyTester [L] UTRL [L]	SAGE [L]

Mechanisms: P prompting; A analysis; R retrieval; F execution feedback;
L parameter learning; S search. Blank cells are unlisted, not absent.

Fig. 2. A representative literature map, based on the methods discussed in this section [2, 4, 9, 13, 16, 18, 27, 36, 45, 55, 57, 62, 70, 86]. A setting can produce several kinds of artifacts, and mechanisms can be combined. Blank cells indicate no example selected for display.

use different scopes and collection protocols. We therefore position our contribution through the connections in Table 1, rather than through a claim of more extensive retrieval.

Table 1 places this account alongside existing reviews. Its intended value is the connection between dimensions: how a feedback source affects oracle independence, how the generated artifact changes the evaluation denominator, and how training supervision constrains claims of autonomy. These connections also provide a common vocabulary for integrating 2026 work on adversarial training, agents, and test maintenance into the earlier literature.

4 Testing Settings and Generated Artifacts

Figure 2 maps representative work to testing settings and generated artifacts. The mechanism labels identify selected operations discussed below; they are illustrative assignments rather than exhaustive coding or publication counts.

4.1 Unit Tests and Repository Context

Unit-test generation is the most visible setting in the reviewed collection. The task appears local because the target is a function or method, but executable tests depend on surrounding types, imports, initialization, and test conventions. AthenaTest established an earlier transformer-based approach that uses focal context and supervised code–test data [73]. TestPilot combines function information and documentation with execution-based repair [62], while ChatUniTest organizes generation, validation, and repair into a framework [10]. These approaches illustrate the transition from generating a plausible test body to constructing an executable artifact in its environment.

CITYWALK addresses project dependencies and C++ knowledge [93], exposing construction, linking, fixtures, and external dependencies absent from standalone-function tasks. Existing tests additionally supply conventions and an improvement baseline in TestGen-LLM [3]. Integration value and the validity of inherited behavioral assumptions remain distinct.

ASTER makes static context and mocking part of test construction for Java and Python, while ChatTester combines generation with an iterative refinement process [54, 88]. KTester distinguishes knowledge used in test design from knowledge used in implementing the test [38]. Across these systems, framework familiarity, dependency resolution, and behavioral selection address different obstacles; an executable test need not be a well-chosen test.

4.2 Requirements, Scenarios, and Acceptance Tests

Requirements-driven generation begins with intended behavior expressed independently of an executable test. RAGTAG retrieves domain information to generate test scenarios from natural-language requirements [4]. Multi-step generation of system-level test specifications uses intermediate artifacts to structure this transformation [50]. Both settings expose an important distinction: producing a useful test design and producing a runnable script are separate accomplishments, with different evaluation methods and human responsibilities.

An industrial acceptance-testing study makes this separation explicit. AutoUAT transforms user stories into Gherkin scenarios, while Test Flow uses those scenarios and page HTML to construct Cypress scripts [18]. Information access changes between stages. The scenario can be judged against a user story; the script also needs knowledge of concrete page structure. APITestGenie similarly combines requirements and API specifications to generate web API tests [56]. These pipelines suggest that traceability should connect the requirement, the scenario, the executable actions, and the assertion, so that a failure can be attributed to the appropriate transformation.

Change intent and industrial tickets add intermediate settings between independent requirements and code-conditioned regression. Testora uses PR intent with code changes, existing tests, and before/after behavior to identify regressions [58]. AToMIC combines JIRA tickets and recent code changes to construct Gherkin scenarios and mobile UI tests in an industrial case study [20]. Intent traceability and executable-script construction should be assessed separately; the availability of natural-language text does not by itself establish implementation-independent generation.

4.3 Oracles, Postconditions, and Metamorphic Relations

Oracle generation isolates the part of a test that decides whether behavior is acceptable. TOGA generates exception and assertion oracles using a neural approach [15]. TOGLL also learns assertion and exception oracles with code-conditioned prompts [24]. TOGA's subsequent large-scale evaluation and replication work demonstrates the importance of examining oracle generation under changed evaluation conditions [25]. An oracle should therefore be assessed by the behavior it accepts and rejects, in addition to whether it resembles a developer-written assertion.

Postconditions and metamorphic relations broaden the oracle representation. The nl2postcond work translates natural-language intent into executable method postconditions [16]. Work on executable metamorphic relations derives relations from requirements and API specifications [64]. A relation can test consistency across executions when predicting one exact output is difficult. Its applicability conditions are part of its meaning: a plausible transformation can be invalid for inputs that violate an unstated precondition. Thus, generating the relation and establishing its domain of validity remain distinct tasks.

Documentation can supply more explicit behavioral constraints. Doc2OracLL examines Javadoc components in oracle generation, whereas Tratto combines source and documentation with constraints on the tokens available for constructing axiomatic Boolean oracles [26, 52]. Documentation selection concerns evidence relevance; grammatical and API constraints concern admissible expressions. Neither operation alone proves that a generated constraint captures the intended contract.

Nexus uses execution-grounded multi-agent processes for oracle synthesis [27], and CANDOR incorporates multi-agent consensus into end-to-end JUnit generation [83]. These approaches

combine test construction and oracle review. Their interaction patterns can expose contradictions, but the source of behavioral evidence still determines what agreement establishes. Agreement among agents or with candidate executions must be interpreted relative to the requirements being tested.

4.4 API and GUI Interaction Sequences

For APIs and GUIs, test generation must maintain state across actions. An endpoint can require a resource identifier returned by an earlier request; a GUI action can depend on a screen reached through navigation. AutoRestTest combines LLM support with multi-agent reinforcement learning for REST API testing [66]. SAINT uses program analysis and LLM-based agents for service-level integration tests [55]. LlamaRestTest adapts smaller models to API dependencies and input values using specifications and server responses, while ARMeta constructs executable metamorphic API tests from specifications [32, 33]. The relevant artifact is therefore a coordinated interaction sequence, including dependencies between operations, rather than a collection of independent input values.

DroidAgent and GPTDroid apply LLM reasoning to mobile GUI exploration and interaction [43, 86]; ScenGen coordinates observation, decision, execution, supervision, and recording around mobile scenarios [87]. Here, observations and actions are interleaved: the next action depends on the current interface. Reaching a screen demonstrates exploration; checking that the screen implements a requirement additionally needs an oracle. This distinction also affects replay. A sequence generated through adaptive interaction may need stable selectors, state initialization, and checkpoints before it becomes a repeatable regression test.

4.5 Fuzzing Inputs and Infrastructure

Fuzzing gives the LLM a different role. TitanFuzz and FuzzGPT generate programs for testing deep-learning libraries, with the latter emphasizing edge cases [13, 14]. Fuzz4All uses LLMs in a broader fuzzing framework [81]. These methods exploit the model's ability to produce structured inputs while relying on execution and external failure detectors to identify anomalies.

The generated object can also be an input generator. ChatAFL extracts protocol-related information to support grammar construction and message-sequence exploration [48]. PromptFuzz generates fuzzing drivers [45]; ELFuzz searches over synthesized fuzzers [9]; Bulbasaur generates customized mutators using branch-oriented information [79]. G2FUZZ synthesizes reusable generators for non-textual inputs such as binary files, extending this distinction beyond generated textual programs [90]. Scheduzz additionally couples API-combination constraints with driver generation and campaign scheduling [41]. This progression changes the cost model because one generated artifact can produce many executions. A survey that records only the number of LLM-generated test cases would miss both the reuse and the contribution of the conventional fuzzing engine.

4.6 Bug Reproduction and Tests for Code Generation

LIBRO generates bug-reproducing tests from natural-language reports [31]. SWT-Bench evaluates test generation in the context of real-world fixes [53]. These settings provide a particular behavioral distinction: the test should expose the issue in a buggy version and accept a fixed version. Reproducing a known failure and discovering an unknown defect are related capabilities but use different information.

Generated tests also influence which code is selected or how it is repaired. CodeT uses generated tests to help choose among candidate programs [8], UTGen learns tests for automated debugging [59], and CodeRM scales test generation for code reward modeling [47]. TiCoder introduces

user feedback on tests to clarify intent during code generation [17]. In all four cases, the downstream outcome depends on both the generator and the consumer of its tests. A useful survey must therefore retain test-level measurements alongside code-selection or repair outcomes.

5 Generation Mechanisms

5.1 Context Construction and Task Decomposition

Prompting determines which parts of a testing problem the model sees at once. Direct prompts usually combine a target, an instruction, and a desired output format. Examples can provide the test framework, naming conventions, and assertion style. Context construction becomes a substantive method contribution when it selects information needed to resolve dependencies or infer the intended behavior. TestPilot and ChatUniTest illustrate focal-code and environment-aware construction [10, 62]; RAGTAG illustrates retrieval of domain material for requirements interpretation [4].

Structural context construction complements retrieval. HITS slices methods to organize generation around smaller code regions; TELPA incorporates program analysis and usage constraints to target difficult coverage; KTester supplies project and testing knowledge [38, 80, 84]. Slicing reduces the local reasoning unit, analysis identifies relationships outside it, and domain knowledge supplies conventions or behavioral rules. Their contributions should be compared at the obstacle each addresses.

Retrieval has two separable functions. It can supply technical facts needed for execution, such as an API signature, or behavioral facts needed for an oracle, such as a business rule. These should be evaluated independently. An increase in compilation success can establish that retrieved API facts were useful without establishing that the retrieved business rule was current or applicable. A retrieval study on ML/DL APIs reports coverage improvements without improved correctness under its evaluation [63]. This provides conditional evidence that technical reach and behavioral judgment can respond differently to added context. Doc2OracLL's documentation analysis and the context comparisons in a Java-oracle empirical study address related questions under different tasks [26, 51]; their outcomes should not be pooled into a universal ranking of retrieval sources.

SymPrompt decomposes prompting around execution paths [61]; requirements pipelines separate interpretation, scenario construction, and scripting [18, 50]. Intermediate artifacts expose omitted scenarios, invalid locators, and wrong expectations as different errors. Their utility depends on final behavior as well as stage-specific success, because an early misunderstanding can propagate.

5.2 Input, State, and Environment Construction

Input construction must satisfy both the external input domain and the execution interface. Grammar-based methods such as SAGE and LogiCase make admissible input structures explicit [2, 67]; repository-oriented approaches must also recover fixtures and dependencies [93]. For API and GUI testing, validity extends to action order and state: a well-formed request can still be unusable before a required resource is created [55, 66, 86]. These mechanisms solve different parts of reaching a meaningful observation.

Exception tests make the setup obligation concrete: exLong conditions specialized generation on throw traces, guards, and existing non-exception tests [89]. ASTER's mocking addresses dependencies that would otherwise block execution [54]. Reaching an exception and choosing whether that exception is expected remain separate decisions.

For the sorting example, preserving a copy of the original input makes mutation observable. Sorted expected values alone omit this setup obligation. Rejected inputs, failed setup, and assertion failures should therefore have separate denominators.

5.3 Oracle Construction and Representation

An oracle can predict an exact result, check a property, relate several executions, or obtain a human judgment. Postconditions and executable metamorphic relations express behavioral constraints [16, 64]; TiCoder obtains user feedback to clarify intended behavior [17]; Nexus and CANDOR use agent interaction in oracle construction [27, 83]. The representation determines which behaviors the test can express, while the evidence source determines why those behaviors should be accepted.

For sorting, ordering, permutation preservation, and unchanged input express the stated obligations. Expecting a particular equal-key order can impose unstated stability. Oracle representation and applicability conditions therefore affect both fault detection and acceptance of legal behavior.

5.4 Program Analysis and Hybrid Search

Program analysis provides explicit structure that the model would otherwise need to infer from text. Dependency information can identify construction requirements; call relationships can constrain operation sequences; coverage can identify unexercised branches. CITYWALK and SAINT illustrate different uses of program structure for executable testing [55, 93]. The source of structure matters: a graph recovered from implementation describes reachable operations, while a behavioral contract specifies what those operations should mean.

CODAMOSA combines search-based test generation with LLM-generated examples when coverage progress stalls [37]. This division of labor is different from asking an LLM to generate an entire suite repeatedly. Search performs repeated local exploration, and the model supplies candidates that may open new regions. Fuzzing-driver and mutator generation extend the same principle to reusable generators [9, 45, 79]. G2FUZZ reuses synthesized generators, while Scheduzz allocates execution resources across campaigns [41, 90]. Model-call cost, artifact lifetime, and conventional search effort jointly determine the contribution of generation.

Hybrid comparisons need matched search engines, instrumentation, stopping rules, and execution budgets. Component ablations should isolate the model's contribution while accounting for conventional computation.

5.5 Execution Feedback and Iterative Repair

Feedback can localize malformed syntax, missing dependencies, or unexercised branches. An assertion failure is ambiguous: the test, implementation, environment, or requirement interpretation may be wrong. The acceptance decision must preserve this distinction.

CoverUp uses coverage and execution information in iterative Python test generation [57]. TestGen-LLM filters proposed improvements through operational criteria that support regression-test integration [3]. MuTAP uses mutation testing to guide generation [12]. These systems demonstrate that feedback can be used for different decisions: repair a candidate, reject it, request another test, or redirect search. Recording only that a system is "execution-guided" loses this distinction.

Repair is an intervention on a particular part of the test. ChatTester and YATE refine near-miss tests, and YATE also discusses revising assertions against observed program behavior [35, 88]. DCAware studies missing-oracle completion for executable regression prefixes and reports a self-repair trap in which iterative repair can favor easier-to-pass assertions [39]. These accounts address different repair operations and settings. Their comparison should retain whether a repair changes syntax, setup, inputs, or the behavioral judgment, and whether the current implementation is assumed to define expected behavior.

For conformance, replacing an expected value with candidate output can erase the violation being tested. Setup repair and oracle revision therefore need different evidence. RESTestBench

Table 2. Representative mechanisms and the interpretation of their feedback. The last column describes the immediate evidence supplied by the signal, not the full set of claims in each paper.

Representative work	Main operation or feedback	Immediate interpretation
SymPrompt [61]	Path-oriented prompt decomposition	Guidance toward a structural target
CODAMOSA [37]	LLM candidates during search plateaus	New material for continued exploration
CoverUp [57]	Coverage and execution feedback	Reachability and artifact execution
MuTAP [12]	Surviving mutants guide prompts	Discrimination within a mutation model
TestGen-LLM [3]	Pass, stability, and coverage filters	Suitability for regression augmentation
Nexus [27]	Candidate execution and agent discussion	Executed behavior plus model interpretation
AdverTest [7]	Test–mutant interaction at inference	Adaptation to the current adversary
Bulbasaur [79]	Branch-guided mutator synthesis	Reusable input exploration capability

makes SUT-feedback quality an explicit evaluation condition [34]. Paired repair branches from the same initial test can measure recovery and degradation, then validate the final suites independently.

5.6 Agents, Collaboration, and Adversarial Generation

Agent architectures assign responsibilities such as planning, code generation, execution, and review to separate components. TestAgent follows a workflow inspired by human testing and incorporates static knowledge and tool use [92]. CANDOR and Nexus emphasize collaboration around test oracles [27, 83]. The analytical question is what separation changes: available information, decision authority, search diversity, or execution budget.

ConVerTest generates code and test candidates and screens them using self-consistency, verification prompts, and dual execution agreement [68]. It avoids requiring a pre-existing reference implementation, while still relying on generated implementations and their executions. VALTEST uses semantic entropy and a learned classifier to assess synthesized test validity [69]. Consensus and predicted validity are additional acceptance signals; their reliability depends on shared errors, training labels, and the evaluation distribution.

Shared models and context can correlate agent errors. A checker querying an independent source contributes different evidence even when it uses the same model. Matched-budget controls should compare multiple roles with one combined-context agent, sampling and selection, and deterministic checking.

AdverTest pairs test and mutant agents in an adversarial generation loop [7]. Such inference-time competition changes the current artifacts. It differs from reinforcement learning that changes model parameters across tasks, discussed in Section 6. Both can expose weaknesses in an existing test suite, but their generalization claims differ. A stronger suite for the current target is evidence about search; a stronger frozen model on new tasks is evidence about learning.

Table 2 links operations to immediate evidence. Information access and acceptance decisions explain their differences more precisely than strategy names alone.

6 Learning Test Generators

6.1 Supervised Learning and Training Data

Training changes the generator's behavior beyond one target or interaction. Supervised learning commonly uses code–test pairs or descriptions paired with tests. AthenaTest provides a historical example of learning unit-test generation with focal context [73]. Such data teach framework syntax and common assertion patterns, but the training label also encodes a particular developer's testing

choices. Multiple valid tests can exist for the same function, and a recorded test can be incomplete or redundant. Exact reproduction of a reference test is consequently a limited measure of learned testing ability.

Training corpora and task-specific adaptation supply complementary evidence. CAT-LM aligns code and test files during pretraining; UniTSyn uses language-server information to extract focal-code/test relationships across languages [22, 60]. FuzzAug expands training data through coverage-guided fuzzing [23]. exLong specializes in exception tests, while TOGLL and Tratto learn oracle construction [24, 52, 89]. These examples distinguish learning repository correspondence, expanding observed inputs, and specializing a behavioral judgment. UniTSyn is principally a training-data resource, rather than another end-task evaluation benchmark.

Data filters determine the supervision: passing examples teach executability; discriminating examples additionally depend on the chosen faulty-program pool. Held-out projects and independently constructed faults test whether training transfers beyond that pool.

6.2 Reinforcement Learning and Reward Meaning

PyTester combines supervised learning and reinforcement learning for text-to-test generation, using automatic signals that include executable reference information [70]. RLSQM learns from automatic feedback associated with unit-test quality [65]. These approaches illustrate why the reward must be described in terms of the property being optimized. Syntax, execution success, structural coverage, maintainability, and semantic validity are related engineering concerns, but a reward for one does not directly establish improvement in another.

CURE makes the supervision distinction particularly clear. Its main co-evolutionary training setting omits ground-truth code supervision but pairs tasks with ground-truth unit tests, which help label generated programs and construct coder/tester rewards [78]. Thus, absence of a reference implementation does not establish absence of reference supervision. Candidate programs, reference tests, and reference code must each be recorded, including changes across ablations.

Training access and inference access describe different dependencies. PyTester can generate from a description while training rewards execute a reference [70]. Reference passing also assumes that the reference satisfies the requirement and that the test ignores incidental behavior.

Table 3 records stage-specific evidence, rather than assigning one black-box label per paper. Rewards should improve detection while constraining rejection of permitted behavior. Reference consistency is a practical proxy when alternative legal implementations are unavailable; the reference assumption remains explicit.

6.3 Adversarial Learning and Self-Play

UTRL learns a test generator through interaction with a code generator and uses reference and sampled programs in its training procedure [36]. ATGen develops adversarial reinforcement learning for test-case generation with a changing adversarial curriculum [40]. Sol-Ver studies self-play for solving and verifying code tasks [42]. TCS separates stages aimed at reference-consistent tests and adversarial testing [82]. CURE jointly evolves coder and tester behavior with reference-test-derived supervision in its main setting [78]. Together, these approaches move test generation from imitating recorded tests toward learning from the behavior of generated programs.

Adaptive difficulty is valuable if it transfers beyond the current opponent. Evaluation should freeze the tester and use held-out tasks, independent error pools, and other code-model families. Matched-budget fixed or refreshed pools isolate interaction from additional erroneous examples. Agreement alone can preserve a shared misunderstanding; requirements, verified constraints, references, or human judgments anchor the labels.

Table 3. Selected information-access configurations. Entries describe the cited configuration or stage; they are not claims about every variant or the underlying model's pretraining data.

Configuration	Generation context	Additional evidence source	Consequence for interpretation
TestGen-LLM [3]	Existing tests and program context	Execution, stability, and coverage checks	Preserves observed regression behavior
AutoUAT / Test Flow [18]	User stories, then scenarios and HTML	Human assessment in the industrial study	Design and implementation stages have different access
Nexus [27]	Task information and agent interaction	Candidate implementation execution	Observations depend on candidate behavior
PyTester [70]	Description and signature at inference	Reference execution during training	Inference access does not describe supervision needs
UTRL [36]	Task-conditioned test generation	Reference and sampled programs during training	Adversarial learning retains a reference dependency
CURE, main training [78]	Task-conditioned code/test generation	Ground-truth tests label generated programs	No ground-truth code does not mean no reference supervision
ConVerTest [68]	Task and generated code/test candidates	Candidate execution and consensus	No pre-existing reference; candidate evidence remains
Requirements-oracle study [46]	Requirements and minimal method structure	Researchers derive requirements from repair differences	Generator access differs from construction access

6.4 Learning Inputs and Learning for Downstream Use

SAGE and LogiCase use grammar-based representations to connect input specifications to generated test inputs [2, 67]. Their contribution concerns admissible and diverse inputs. An evaluator still needs a mechanism for deciding whether the resulting program behavior is correct. This division can be valuable because input validity has a different structure from expected-output reasoning and can sometimes be checked mechanically.

UTGen targets tests useful for debugging, whereas CodeRM trains and scales tests to support code reward modeling [47, 59]. These objectives suggest an additional evaluation layer: the same test generator should be assessed both as a testing component and inside its intended downstream workflow. Improved code selection can arise from a better ranking signal even if some tests remain incorrect. Conversely, a semantically correct but weak test suite may have little effect on selection among already plausible candidates. Reporting both layers explains how the generated tests contribute to the final outcome.

7 Benchmarks and Quality Evidence

7.1 A Quality Profile Rather Than a Single Score

We organize test quality as a profile of complementary properties. *Input validity* asks whether the input satisfies the task's domain constraints. *Executability* asks whether the test can be built and run in the intended environment. *Oracle correctness* asks whether its judgment follows the intended specification. *Defect detection* asks whether it rejects relevant invalid behavior. *Non-rejection* asks

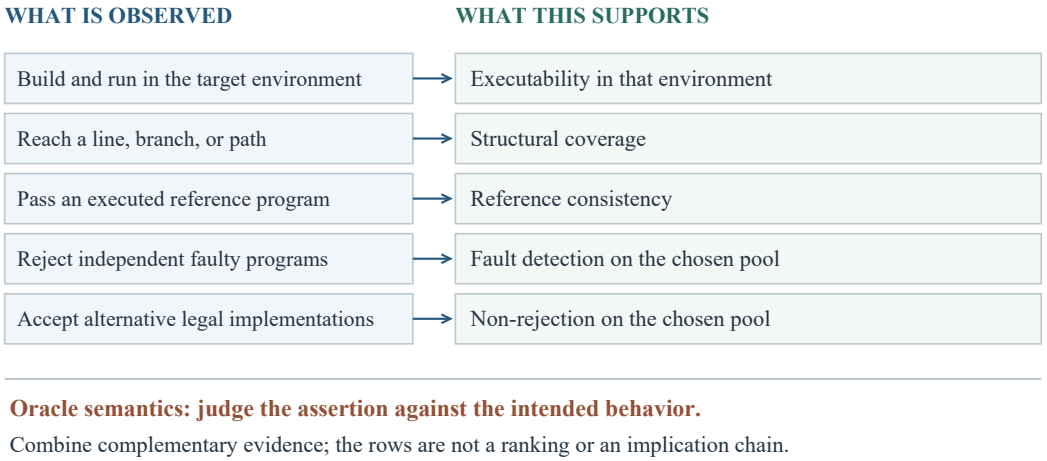


Fig. 3. Quality evidence supports different conclusions. Each arrow connects an observation to its immediate interpretation. Oracle correctness additionally depends on the intended specification and the validity of the evidence sources. These dimensions are complementary rather than successive levels of one score.

whether it accepts permitted behavior. Coverage, stability, maintainability, and cost add further dimensions needed for deployment.

Denominators differ: compilation can cover all attempts, whereas coverage and mutation often use retained tests. Report attempted, executable, accepted, and retained artifacts to expose filtering. Per-test and per-suite results are complementary, since one invalid assertion can cause an otherwise useful suite to reject a correct implementation.

Figure 3 connects each observation to its bounded interpretation. Coverage measures reach; mutation depends on its fault model and test validity; reference passing assumes a suitable reference. Assertion matching measures similarity, code-selection accuracy measures downstream utility, and repeated-run passing measures stability. These signals remain useful at their respective levels.

7.2 Benchmark Families

TestGenEval evaluates test generation and completion in real Python repositories [30]. TESTEVAL emphasizes test generation for program coverage, including targeted structural objectives [76]. Both are relevant to executable testing, but their task construction changes the context and the question being measured. Repository tasks expose dependencies and established tests; structural targets isolate whether generation can reach particular code regions.

SWT-Bench evaluates tests distinguishing buggy software from real-world fixes [53]. Patch discrimination requires issue-related failure and acceptance of the fix, but does not characterize every legal implementation.

ULT/PLT studies test generation from real-world functions and examines contamination-related evaluation conditions [29]. RESTestBench studies REST API tests generated from natural-language requirements, including requirement precision and SUT feedback [34]. These benchmarks expose issues that small function tasks can underrepresent: complex context, ambiguity, dependencies between operations, and the quality of feedback used to revise a test.

Recent empirical work adds complementary evaluation dimensions. A study of real Python bugs compares LLM and human unit tests through fault detection [74]. Research under software evolution studies test generation across semantic changes [21]. Database testing work investigates

Table 4. Benchmark selection should follow the intended claim. Rows identify primary emphases rather than exhaustive benchmark capabilities.

Benchmark family	Useful question	Comparison condition to preserve
TestGenEval [30]	Can tests be constructed in repositories?	Existing tests, dependencies, generation versus completion
TESTEVAL [76]	Can tests reach structural targets?	Target visibility and coverage denominator
SWT-Bench [53]	Can tests expose an issue and accept its fix?	Issue information and buggy/fixed execution protocol
ULT/PLT [29]	How do real functions and leakage conditions affect evaluation?	Dataset construction and model exposure assumptions
RESTestBench [34]	How do requirements and feedback affect API tests?	Requirement precision and feedback validity
TC-Bench [44]	Can tests distinguish selected erroneous solutions?	Candidate-pool construction and matrix definition
MultiFileTest [77]	Can tests use multi-file project context?	Language, repository context, and repair configuration
TOGBench [72]	Can generated oracles replace diverse Java oracles?	Oracle type, reinsertion, and metric definitions

flakiness in industrial and open-source settings [6]. These studies broaden the target from initial generation quality to the behavior of tests on historical defects, changed software, and repeated executions.

Newer resources isolate further comparison conditions. TC-Bench selects a compact pool of erroneous contest programs through a code–test outcome matrix [44]. Its discrimination and rank properties concern that observed pool. MultiFileTest moves evaluation to multi-file projects and distinguishes error-fixing configurations [77]. TOGBench covers several oracle forms and reinserts generated oracles into Java tests for compilation, execution, and mutation assessment [72]. The latter's repository uses "false positive" for erroneous programs that still pass; we retain that operational definition rather than equating it with rejection of legal implementations.

The benchmark portfolio should follow the claim: repository usability, requirements reasoning, patch discrimination, and independent discovery require different evidence. Scores from unrelated protocols cannot be combined into a method ranking.

7.3 Implementation State and Metric Interpretation

Empirical studies increasingly separate tests generated from correct and buggy code. Work on incorrect-code influence and buggy-code misguidance examines how implementation errors affect generated judgments [28, 95]. Replacing code with a generated docstring can change that influence, but the docstring remains code-derived [95]. The requirements-oracle study instead evaluates requirement agreement separately from agreement with the SUT; its ten Lang-bug cases and repair-derived requirement construction bound the setting [46].

The same distinction qualifies proxy metrics. A replicability study reports that coverage and mutation can inform model comparison in regression settings where code is reasonably assumed correct, while their reliability for detecting existing bugs is weaker [94]. Test Wars also illustrates that mutation, coverage, and fault-detection rankings can differ across LLM, search-based, and symbolic

approaches [1]. These observations support task-specific metric interpretation, rather than discarding structural metrics. The Java-oracle study uses developer-oracle comparison as a correctness proxy and mutation as a detection proxy [51]; neither exhausts specification conformance.

Fuzzing adds a related failure-attribution problem. A crash from a generated driver can reflect an invalid API use rather than a reachable product defect. Google's OSS-Fuzz-Gen report examines false-positive mitigation using API constraints and realistic entry-point context [49]. Valid input or driver construction, reproduced failure, and confirmed defect should therefore be separate stages of the reporting funnel.

7.4 Independent Oracle Assessment

The requirement can be viewed as defining a set of permitted implementations. For an illustrative evaluation, let $\mathcal{P}^+$ be independently validated implementations that satisfy the requirement and let $\mathcal{P}^-$ be independently constructed faulty implementations. For a test suite T, define

$$\text{RejectLegal}(T) = \frac{\sum_{p \in \mathcal{P}^+} \mathbf{1}[T(p) = \text{fail}]}{|\mathcal{P}^+|}, \qquad \text{DetectFault}(T) = \frac{\sum_{p \in \mathcal{P}^-} \mathbf{1}[T(p) = \text{fail}]}{|\mathcal{P}^-|}. \quad (1)$$

The two quantities measure complementary behavior on finite evaluation pools. They require a nonempty pool of each kind, valid execution environments, and trustworthy pool labels. They are an evaluation recommendation rather than measurements extracted from the reviewed studies. Reporting them together exposes suites that detect many faults by also rejecting permitted behavior. Neither quantity alone proves specification-wide correctness.

Alternative correct algorithms and permissible output choices test whether an oracle encodes unstated implementation details. Faults withheld from training, prompting, and feedback test independent detection. Where executable pools are unavailable, expert scenario assessment should record traceability, applicability, expected behavior, omissions, and assessor disagreement; textual similarity alone does not establish detection.

7.5 Comparability, Leakage, and Cost

Comparisons need matched information contracts: requirements-only generation, reference access, and candidate-feedback repair are different settings. Training data, retrieval, existing tests, and evaluation artifacts can cross intended boundaries. Contamination controls should identify the paths they address.

Cost should include model calls, input and output tokens, conventional executions, environment preparation, and human intervention. For trained generators, training cost and deployment cost answer different questions and should be reported separately. For hybrid fuzzers, execution throughput and the reuse of generated infrastructure matter. For agent systems, total calls across all roles matter. An equal-token comparison and an equal-wall-time comparison may favor different designs, so both can be informative when resources permit.

Generation randomness and execution nondeterminism are separate sources of variance. Project-level results, repeated generation, repeated execution, failed attempts, and stopping rules make uncertainty and selection effects assessable.

8 Practical Integration and Research Agenda

8.1 Choose a Workflow from the Testing Objective

Workflow selection starts with the behavioral decision, available evidence, and missing artifact. Regression augmentation can use existing tests and coverage, as in TestGen-LLM [3]. Conformance

needs requirement-backed judgments; fuzzing needs valid generators and failure attribution; interactive code generation can use user-confirmed examples, as in TiCoder [17]. Assertion provenance connects these choices to review and maintenance.

8.2 Industrial Adoption and Human Revision

Industrial evidence includes different acceptance decisions. TestGen-LLM studies improvement of existing tests through operational filtering [3]; the acceptance-testing case study evaluates both generated scenarios and their conversion into executable scripts [18]. RAGTAG and system-level test-specification generation additionally involve domain interpretation and assessment [4, 50]. ASTER adds dependency handling and mocking, while AToMIC connects industrial tickets to mobile acceptance scripts [20, 54]. The OSS-Fuzz-Gen report adds failure-validation work to the adoption burden [49]. A scenario judged useful by an engineer and a script accepted into a regression workflow are distinct outcomes.

A deployment evaluation should retain these stages: generated, reviewed, revised, executed, and adopted. Human effort includes interpreting requirements, supplying missing context, correcting assertions, and maintaining the accepted test. Reporting the time and rejection reasons at each stage would connect the artifact's technical quality to its actual workflow value. The reviewed examples motivate these dimensions; this synthesis does not pool their heterogeneous adoption measures.

8.3 Oracle Provenance and Independent Validation

Linking an assertion to a requirement clause, validated example, formal property, or regression observation could guide acceptance and repair. The research question is whether checked provenance improves oracle quality and maintenance. A matched-budget comparison should measure unsupported assertions, rejection of legal implementations, independent detection, and review time. Plausible explanations without verified evidence links do not establish that benefit.

8.4 Feedback That Preserves Valid Tests

Repair should recover invalid tests while preserving valid judgments. Branching from the same initial artifact allows separate measurement of recovery and regression, followed by independent suite evaluation. Controls can compare setup-only repair, unrestricted repair, no semantic repair, and trusted-reference checking. The information and budget must match; improved passing alone cannot determine whether assertions were corrected or weakened.

8.5 Generalization in Learned and Adversarial Testers

Adversarial curricula offer difficult errors [36, 40, 82], but transferable capability requires a frozen tester evaluated on new opponents, repositories, and specification styles. Suite rewards also obscure which tests help or mislead. Comparing aggregate rewards with verified test-level feedback can test the value of finer credit assignment, while retaining validity constraints and independent detection.

8.6 Repository Scale and Lifecycle Evaluation

TestGenEval, CITYWALK, and service-level testing provide repository-oriented starting points [30, 55, 93]. Lifecycle evaluation should additionally label whether a change preserves semantics or deliberately changes requirements. Evolution and database-flakiness studies motivate retaining prior tests and measuring execution stability, maintenance effort, and unintended-regression detection [6, 21]. Version-aware retrieval should be compared with unrestricted context reuse, since existing tests can preserve brittle setup or obsolete obligations.

8.7 Reproducible Evidence and Efficient Allocation

Reproducibility packages should expose the path from request to retained suite. Relevant records include model and prompt versions, context selection, generated candidates, tool feedback, repairs, rejection reasons, and final evaluation inputs. This enables analysis of why one method succeeds and whether its filters hide a large failure burden. For training methods, data construction and reward implementation are equally important.

Budget allocation is another cross-cutting research question. A fixed budget can be spent on generating more candidates, checking oracles, executing additional faults, or resolving uncertain requirements. Dynamic scaling in CodeRM [47] and reusable fuzzing infrastructure [9, 45, 79] illustrate different responses to this allocation problem. A broader study could compare these investments under a common deployment objective, such as maximizing independent detection while bounding invalid-suite rate and latency.

9 Threats to Validity

The collection supports a qualitative map of research directions rather than estimates of their prevalence. Web-based discovery, English-language emphasis, and the stated source restrictions can omit relevant studies. Performance, accessibility, concurrency, and distributed-system testing received less focused coverage than functional testing and fuzzing. Selected neural predecessors provide context but do not constitute a complete history of learned test generation.

Inspection depth varies: abstract-level evidence supports broad mechanism descriptions; context permissions, reward definitions, and denominators require fuller coding. We do not pool numerical gains. Multi-database retrieval, complete variant extraction, and independent coding remain necessary for a systematic extension. The framework has not undergone inter-rater validation, and multi-stage methods can occupy several categories. Preprint revisions, changing model services, and publication bias further limit the dated synthesis.

10 Conclusion

LLM-based test generation spans scenarios, executable suites, oracles, and reusable fuzzing infrastructure. We connected these artifacts to their information sources, generation mechanisms, and evaluation assumptions. The synthesis separates executable construction from oracle justification: coverage, reference consistency, patch discrimination, independent detection, and acceptance of legal behavior supply complementary evidence. Making information access, evidence provenance, and cost explicit supports more interpretable comparisons and guides research on generators that remain useful across repositories, adversaries, and software changes.

Data Availability

The accompanying materials comprise the 95-record source register, search and inspection notes, supplemental provenance, synthesis outline, bibliography, manuscript sources, and editable conceptual figures. A public development repository contains these materials; an anonymous review-access endpoint remains to be configured before submission. The register preserves primary-source links and inspection-depth annotations. No experimental dataset or reproduced performance results are claimed.

References

[1] Azat Abdullin, Pouria Derakhshanfar, and Annibale Panichella. 2025. Test Wars: A Comparative Study of SBST, Symbolic Execution, and LLM-Based Approaches to Unit Test Generation. arXiv:2501.10200

[2] Aditi, Hyunwoo Park, Sicheol Sung, Yo-Sub Han, and Sang-Ki Ko. 2025. SAGE: Specification-Aware Grammar Extraction for Automated Test Case Generation with LLMs. arXiv:2506.11081

[3] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. arXiv:2402.09171

[4] Chetan Arora, Tomas Herda, and Verena Homm. 2024. Generating Test Scenarios from NL Requirements using Retrieval-Augmented LLMs: An Industrial Study. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 240–251. doi:10.1109/RE59067.2024.00031

[5] Cristian Augusto, Antonia Bertolino, Guglielmo De Angelis, Francesca Lonetti, and Jesús Morán. 2025. Large Language Models for Software Testing: A Research Roadmap. arXiv:2509.25043

[6] Alexander Berndt, Thomas Bach, Rainer Gemulla, Marcus Kessel, and Sebastian Baltes. 2026. On the Flakiness of LLM-Generated Tests for Industrial and Open-Source Database Management Systems. arXiv:2601.08998

[7] Pengyu Chang, Yixiong Fang, Silin Chen, Yuling Shi, Beijun Shen, and Xiaodong Gu. 2026. Test vs Mutant: Adversarial LLM Agents for Robust Unit Test Generation. arXiv:2602.08146

[8] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. arXiv:2207.10397

[9] Chuyang Chen, Brendan Dolan-Gavitt, and Zhiqiang Lin. 2025. ELFuzz: Efficient Input Generation via LLM-driven Synthesis Over Fuzzer Space. In *34th USENIX Security Symposium*. USENIX Association, Seattle, WA, 6279–6298. https://www.usenix.org/conference/usenixsecurity25/presentation/chen-chuyang

[10] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: A Framework for LLM-Based Test Generation. arXiv:2305.04764

[11] Bei Chu, Yang Feng, Kui Liu, Zhaoqiang Guo, Yichi Zhang, Hange Shi, Zifan Nan, and Baowen Xu. 2025. Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities. arXiv:2511.21382

[12] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2023. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. arXiv:2308.16557

[13] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2022. Large Language Models are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. arXiv:2212.14834

[14] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large Language Models are Edge-Case Fuzzers: Testing Deep Learning Libraries via FuzzGPT. arXiv:2304.02014

[15] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A Neural Method for Test Oracle Generation. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2130–2141. doi:10.1145/3510003.3510141

[16] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2023. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? arXiv:2310.01831

[17] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. arXiv:2404.10100

[18] Margarida Ferreira, Luis Viegas, Joao Pascoal Faria, and Bruno Lima. 2025. Acceptance Test Generation with Large Language Models: An Industrial Case Study. In *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 1–11. doi:10.1109/AST66626.2025.00007

[19] Orimoloye Folorunsho and Hassan Reza. 2026. AI-Driven Test Case Generation from Natural Language Requirements: A Survey of Techniques and Research Gaps. arXiv:2606.06563

[20] Pedro Luís Fonseca, Bruno Lima, and João Pascoal Faria. 2025. Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. doi:10.1109/ASE63991.2025.00273

[21] Sabaat Haroon, Mohammad Taha Khan, and Muhammad Ali Gulzar. 2026. Evaluating LLM-Based Test Generation Under Software Evolution. arXiv:2603.23443

[22] Yifeng He, Jiabo Huang, Yuyang Rong, Yiwen Guo, Ethan Wang, and Hao Chen. 2024. UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing. arXiv:2402.03396

[23] Yifeng He, Jicheng Wang, Yuyang Rong, and Hao Chen. 2025. FuzzAug: Data Augmentation by Coverage-guided Fuzzing for Neural Test Generation. https://aclanthology.org/2025.findings-emnlp.847/

[24] Soneya Binta Hossain and Matthew Dwyer. 2024. TOGLL: Correct and Strong Test Oracle Generation with LLMs. doi:10.1109/ICSE55347.2025.00098

[25] Soneya Binta Hossain, Antonio Filieri, Matthew B. Dwyer, Sebastian Elbaum, and Willem Visser. 2023. Neural-Based Test Oracle Generation: A Large-scale Evaluation and Lessons Learned. arXiv:2307.16023

[26] Soneya Binta Hossain, Raygan Taylor, and Matthew Dwyer. 2024. Doc2OracLL: Investigating the Impact of Documentation on LLM-based Test Oracle Generation. doi:10.1145/3729354

[27] Dong Huang, Mingzhe Du, Jie M. Zhang, Zheng Lin, Meng Luo, Qianru Zhang, and See-Kiong Ng. 2025. Nexus: Execution-Grounded Multi-Agent Test Oracle Synthesis. arXiv:2510.26423

[28] Dong Huang, Jie M. Zhang, Mark Harman, Mingzhe Du, and Heming Cui. 2024. Measuring the Influence of Incorrect Code on Test Generation. arXiv:2409.09464

[29] Dong Huang, Jie M. Zhang, Mark Harman, Qianru Zhang, Mingzhe Du, and See-Kiong Ng. 2025. Benchmarking LLMs for Unit Test Generation from Real-World Functions. arXiv:2508.00408

[30] Kush Jain, Gabriel Synnaeve, and Baptiste Rozière. 2024. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. arXiv:2410.00752

[31] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2022. Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. arXiv:2209.11515

[32] Shehroz Khan, Abdullah Mughees, Gaadha Sudheerbabu, Tanwir Ahmad, and Dragos Truscan. 2026. Multi-Agent LLM-based Metamorphic Testing for REST APIs. arXiv:2605.28321

[33] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. LlamaRestTest: Effective REST API Testing with Small Language Models. arXiv:2501.08598

[34] Leon Kogler, Stefan Hangler, Maximilian Ehrhart, Benedikt Dornauer, Roland Wuersching, and Peter Schrammel. 2026. RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements. arXiv:2604.25862

[35] Michael Konstantinou, Renzo Degiovanni, Jie M. Zhang, Mark Harman, and Mike Papadakis. 2025. YATE: The Role of Test Repair in LLM-Based Unit Test Generation. arXiv:2507.18316

[36] Dongjun Lee, Changho Hwang, and Kimin Lee. 2026. Learning to Generate Unit Test via Adversarial Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2026/hash/efb02f96766a3b599c76852abf4d42dd-Abstract-Conference.html

[37] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *Proceedings of the International Conference on Software Engineering (ICSE)*. https://www.microsoft.com/en-us/research/publication/codamosa-escaping-coverage-plateaus-in-test-generation-with-pre-trained-large-language-models/

[38] Anji Li, Mingwei Liu, Zhenxi Chen, Zheng Pei, Zike Li, Dekun Dai, Yanlin Wang, and Zibin Zheng. 2025. Knowledge Matters: Injecting Project and Testing Knowledge into LLM-based Unit Test Generation. doi:10.1145/3744916.3787769

[39] Kefan Li, Hongyue Yu, and Yuan Yuan. 2026. Escaping the Self-Repair Trap: Improving Test Oracle Generation via Dual-Context Awareness. arXiv:2608.05917

[40] Qingyao Li, Xinyi Dai, Weiwen Liu, Xiangyang Li, Yasheng Wang, Ruiming Tang, Yong Yu, and Weinan Zhang. 2026. ATGen: Adversarial Reinforcement Learning for Test Case Generation. In *International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2026/hash/eed25c037bc08afcbefab6f7a6b700e0-Abstract-Conference.html

[41] Yan Li, Wenzhang Yang, Yuekun Wang, Jian Gao, Shaohua Wang, Yinxing Xue, and Lijun Zhang. 2025. Scheduzz: Constraint-based Fuzz Driver Generation with Dual Scheduling. arXiv:2507.18289

[42] Zi Lin, Sheng Shen, Ilia Kulikov, Jingbo Shang, Jason Weston, and Yixin Nie. 2025. Learning to Solve and Verify: A Self-Play Framework for Code and Test Generation. arXiv:2502.14948

[43] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. arXiv:2310.15780

[44] Xianzhen Luo, Jinyang Huang, Wenzhen Zheng, Qingfu Zhu, Mingzheng Xu, Yiheng Xu, Yuantao Fan, and Wanxiang Che. 2025. How Many Code and Test Cases Are Enough? Evaluating Test Cases Generation from a Binary-Matrix Perspective. arXiv:2510.08720

[45] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. Prompt Fuzzing for Fuzz Driver Generation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. ACM, 3793–3807. doi:10.1145/3658644.3670396

[46] Tiancheng Ma and Nasir U. Eisty. 2026. From Business Requirements to Test Assertions: Evaluating LLM-Generated Oracles on Real Bugs. arXiv:2607.10277

[47] Zeyao Ma, Xiaokang Zhang, Jing Zhang, Jifan Yu, Sijia Luo, and Jie Tang. 2025. Dynamic Scaling of Unit Tests for Code Reward Modeling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 6917–6935. doi:10.18653/v1/2025.acl-long.343

[48] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large Language Model guided Protocol Fuzzing. In *Proceedings 2024 Network and Distributed System Security Symposium*. Internet Society. doi:10.14722/ndss.2024.24556

[49] Jonathan Metzman, Paschal C. Amusuo, Ricardo Calvo, Oliver Chang, James C. Davis, and Dongge Liu. 2026. Lessons from Mitigating False Positives in Google's OSS-Fuzz-Gen. https://research.google/pubs/lessons-from-mitigating-false-positives-in-googles-oss-fuzz-gen/

[50] Dragan Milchevski, Gordon Frank, Anna Hätty, Bingqing Wang, Xiaowei Zhou, and Zhe Feng. 2025. Multi-Step Generation of Test Specifications using Large Language Models for System-Level Requirements. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*. Association for Computational Linguistics, 132–146. doi:10.18653/v1/2025.acl-industry.11

[51] Davide Molinelli, Luca Di Grazia, Alberto Martin-Lopez, Michael D. Ernst, and Mauro Pezzè. 2025. Do LLMs Generate Useful Test Oracles? An Empirical Study with an Unbiased Dataset. https://homes.cs.washington.edu/~mernst/pubs/neurosymbolic-oracles-ase2025.pdf

[52] Davide Molinelli, Alberto Martin-Lopez, Elliott Zackrone, Beyza Eken, Michael D. Ernst, and Mauro Pezzè. 2025. Tratto: A Neuro-Symbolic Approach to Deriving Axiomatic Test Oracles. arXiv:2504.04251

[53] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. arXiv:2406.12952

[54] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2024. ASTER: Natural and Multi-language Unit Test Generation with LLMs. arXiv:2409.03093

[55] Rangeet Pan, Raju Pavuluri, Ruikai Huang, Rahul Krishna, Tyler Stennett, Alessandro Orso, and Saurabh SInha. 2025. SAINT: Service-level Integration Test Generation with Program Analysis and LLM-based Agents. arXiv:2511.13305

[56] André Pereira, Bruno Lima, and João Pascoal Faria. 2026. APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs. arXiv:2604.02039

[57] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919. doi:10.1145/3729398

[58] Michael Pradel. 2025. Testora: Using Natural Language Intent to Detect Behavioral Regressions. arXiv:2503.18597

[59] Archiki Prasad, Elias Stengel-Eskin, Justin Chih-Yao Chen, Zaid Khan, and Mohit Bansal. 2025. Learning to Generate Unit Tests for Automated Debugging. arXiv:2502.01619

[60] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM: Training Language Models on Aligned Code And Tests. arXiv:2310.01602

[61] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A study of Coverage Guided Test Generation in Regression Setting using LLM. arXiv:2402.00097

[62] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. arXiv:2302.06527

[63] Jiho Shin, Nima Shiri Harzevili, Reem Aleithan, Hadi Hemmati, and Song Wang. 2024. Retrieval-Augmented Test Generation: How Far Are We? arXiv:2409.12682

[64] Seung Yeob Shin, Fabrizio Pastore, Domenico Bianculli, and Alexandra Baicoianu. 2024. Towards Generating Executable Metamorphic Relations Using Large Language Models. arXiv:2401.17019

[65] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation. arXiv:2310.02368

[66] Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. AutoRestTest: A Tool for Automated REST API Testing Using LLMs and MARL. arXiv:2501.08600

[67] Sicheol Sung, Aditi, Dogyu Kim, Yo-Sub Han, and Sang-Ki Ko. 2025. LogiCase: Effective Test Case Generation from Logical Description in Competitive Programming. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 7742–7750. doi:10.24963/ijcai.2025/861

[68] Hamed Taherkhani, Alireza DaghighFarsoodeh, Mohammad Chowdhury, Hung Viet Pham, and Hadi Hemmati. 2026. Consistency Meets Verification: Enhancing Test Generation Quality in Large Language Models Without Ground-Truth Solutions. arXiv:2602.10522

[69] Hamed Taherkhani, Jiho Shin, Muhammad Ammar Tahir, Md Rakib Hossain Misu, Vineet Sunil Gattani, and Hadi Hemmati. 2024. Toward Automated Validation of Language Model Synthesized Test Cases using Semantic Entropy. arXiv:2411.08254

[70] Wannita Takerngsaksiri, Rujikorn Charakorn, Chakkrit Tantithamthavorn, and Yuan-Fang Li. 2025. PyTester: Deep Reinforcement Learning for Text-to-Testcase Generation. *Journal of Systems and Software* 224 (2025), 112381. doi:10.1016/j.jss.2025.112381

[71] Murat Tasarsu, Ahmet Vedat Tokmak, and Cagatay Catal. 2026. Test case generation using large language models: a systematic literature review. *Cluster Computing* 29, 4, Article 227 (2026). doi:10.1007/s10586-026-06021-z

[72] Tasfia Tasnim, Matthew B. Dwyer, and Soneya Binta Hossain. 2026. TOGBench: A Developer-Written Multi-Variant Dataset and Benchmark Suite for Test Oracle Generation. doi:10.1145/3805760.3814927

[73] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit Test Case Generation with Transformers and Focal Context. arXiv:2009.05617

[74] Phouvadeth Vathana, Prapti Bhatt, Rishi Patel, and Nasir U. Eisty. 2026. LLM vs. Human Unit Tests: Fault Detection on Real Python Bugs. arXiv:2606.08588

[75] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software Testing with Large Language Models: Survey, Landscape, and Vision. arXiv:2307.07221

[76] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 3547–3562. doi:10.18653/v1/2025.findings-naacl.197

[77] Yibo Wang, Congying Xia, Wenting Zhao, Jiangshu Du, Chunyu Miao, Zhongfen Deng, Philip S. Yu, and Chen Xing. 2026. MultiFileTest: A Multi-File-Level LLM Unit Test Generation Benchmark and Impact of Error Fixing Mechanisms. https://aclanthology.org/2026.findings-acl.1403/

[78] Yinjie Wang, Ling Yang, Ye Tian, Ke Shen, and Mengdi Wang. 2025. Co-Evolving LLM Coder and Unit Tester via Reinforcement Learning. arXiv:2506.03136

[79] Yiyi Wang, Dongsong Yu, Ruiqi Dong, Yiyang Chen, Xiaogang Zhu, and Chao Zhang. 2026. Bulbasaur: Branch-Guided Online Mutator Generation for Greybox Fuzzing. In *35th USENIX Security Symposium*. USENIX Association, Baltimore, MD, 4089–4108. https://www.usenix.org/conference/usenixsecurity26/presentation/wang-yiyi

[80] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. arXiv:2408.11324

[81] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2023. Fuzz4All: Universal Fuzzing with Large Language Models. arXiv:2308.04748

[82] Jiacheng Xu, Wentao Zhang, Zhiyi Lyu, Fuxiang Zhang, Chaojie Wang, Yang Liu, and Bo An. 2026. Two-Stage Reinforcement Learning for Sound and Adversarial Test Generation in Code LLMs. arXiv:2609.03955

[83] Qinghua Xu, Guancheng Wang, Lionel Briand, and Kui Liu. 2025. Hallucination to Consensus: Multi-Agent LLMs for End-to-End JUnit Test Generation. arXiv:2506.02943

[84] Chen Yang, Junjie Chen, Bin Lin, Ziqi Wang, and Jianyi Zhou. 2024. Advancing Code Coverage: Incorporating Program Analysis with Large Language Models. arXiv:2404.04966

[85] Zhenzhen Yang, Rubing Huang, Chenhui Cui, Nan Niu, and Dave Towey. 2025. Requirements-Based Test Generation: A Comprehensive Survey. arXiv:2505.02015

[86] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous Large Language Model Agents Enabling Intent-Driven Mobile GUI Testing. arXiv:2311.08649

[87] Shengcheng Yu, Yuchen Ling, Chunrong Fang, Quan Zhou, Yi Zhao, Chunyang Chen, Shaomin Zhu, and Zhenyu Chen. 2025. Scenario-Guided LLM-based Mobile App GUI Testing. arXiv:2506.05079

[88] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. https://mingwei-liu.github.io/assets/pdf/FSE24_chatTester_cameraReady.pdf

[89] Jiyang Zhang, Yu Liu, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2024. exLong: Generating Exceptional Behavior Tests with Large Language Models. arXiv:2405.14619

[90] Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. 2025. Low-Cost and Comprehensive Non-textual Input Fuzzing with LLM-Synthesized Input Generators. https://www.usenix.org/conference/usenixsecurity25/presentation/zhang-kunpeng

[91] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. arXiv:2506.15227

[92] Quanjun Zhang, Ye Shang, Siqi Gu, Jianyi Zhou, Chunrong Fang, Zhenyu Chen, and Liang Xiao. 2026. Multi-Agent LLM Collaboration for Unit Test Generation via Human-Testing-Inspired Workflows. arXiv:2607.09101

[93] Yuwei Zhang, Qingyuan Lu, Kai Liu, Wensheng Dou, Jiaxin Zhu, Li Qian, Chunxi Zhang, Zheng Lin, and Jun Wei. 2025. CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge. arXiv:2501.16155

[94] Junda Zhao, Shurui Zhou, and Eldan Cohen. 2026. Do Coverage and Mutation Scores of LLM-Generated Test Suites Correlate with Their Effectiveness? (Replicability Study). doi:10.1145/3832093

[95] Junda Zhao, Shurui Zhou, and Eldan Cohen. 2026. Evaluating and Mitigating the Misguidance Effect of Buggy Code in LLM-Generated Unit Tests. doi:10.1145/3832204